Computer Graphics in OpenGL
INM376 / IN3005

Module Leader – Dr Eddie Edwards

Lecture 02: Primitives and matrices

Recap from last time

We discussed different approaches to computer graphics (rasterisation, ray tracing, radiosity, volume rendering) and why rasterisation is typically used in computer games.

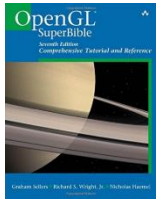
We reviewed the modern OpenGL graphics pipeline, describing how objects in the 3D scene are rendered to form an image.

We covered the mathematics of vectors.

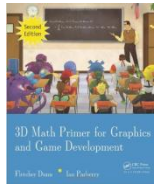
We looked at a first OpenGL program that renders a triangle and you modified it to render stacks of triangles that interactively move in 3D space.

Reminder

Recommended books for the module

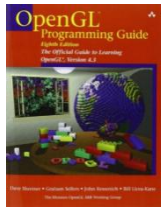


OpenGL Superbible, (Seventh Edition) Sellers et al.



3D Math Primer for Graphics and Game Development, (Second Edition) Dunn and Parberry. Note: Free online access through library!

OpenGL 4 Shading Language Cookbook, (Second Edition) Wolff. Note: Free online access through library!



OpenGL Programming Guide: The Official Guide to Learning OpenGL, Shreiner et al.

Lab 1 Solution

```
void Game::DrawTriangleStack(glm::vec3 s)
{
    for (int i = 0; i < 10; i++)
        DrawTriangle(s+glm::vec3(0, 0, 0 + m_spacing*i));
}
```

Draws a stack of 10 triangles

```
void Game::DrawTriangle(glm::vec3 t)
{
    // Bind the VAO
    glBindVertexArray(m_uiVAO);

    // Set the modeling matrix
    *m_pModelMatrix = glm::translate(glm::mat4(1), t);
    m_pShaderProgram->SetUniform("modelMatrix", m_pModelMatrix);
    m_pShaderProgram->SetUniform("t", (float)m_pTimer->Elapsed());

    // Render the triangle consisting of 3 vertices
    glDrawArrays(GL_TRIANGLES, 0, 3);
}
```

Instructs GPU to use the triangle VAO

Creates a translation matrix

Passes the matrix to the vertex shader

Passes the elapsed time to the vertex shader

Draws the triangle using the matrix and elapsed time

```
// Render method runs repeatedly in a loop
void Game::Render()
{
    // Clear the buffer for rendering a new frame
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    //DrawTriangle(glm::vec3(0, 5, 0));
    DrawTriangleStack(glm::vec3(-5, 0, 0));
    DrawTriangleStack(glm::vec3(0, 0, 0));
    DrawTriangleStack(glm::vec3(5, 0, 0));

    // Swap buffers to show the rendered image
    SwapBuffers(gameWindow.Hdc());
}
```

Clears the buffers

Draws three stacks of triangles

Displays the rendered image

What are those shaders doing?

Vertex shader

```
#version 400

uniform mat4 modelMatrix;
uniform mat4 viewMatrix;
uniform mat4 projectionMatrix;

uniform float t;
layout (location = 0) in vec3 inPosition;
layout (location = 1) in vec3 inColour;

out vec3 vColour;

void main()
{
   gl_Position = projectionMatrix * viewMatrix *
                 modelMatrix * vec4(inPosition, 1.0);
   vColour = inColour;
}
```

- Indicates the version of GLSL (4.0)
- Matrices used to transform points coming into the shader from the client program
- Elapsed time coming into the shader
- Vertex positions and colours coming into the shader
- Output colour sent to the fragment shader
- Transform the vertex
- Pass the colour to the fragment shader

What are those shaders doing?

Fragment shader

```
#version 400

in vec3 vColour;
out vec4 outputColour;

void main()
{
   outputColour = vec4(vColour, 1.0);
}
```

← Indicates the version of GLSL (4.0)

← Colour passed in from vertex shader

← Colour used to render the fragment

← Set the fragment colour to the colour passed in

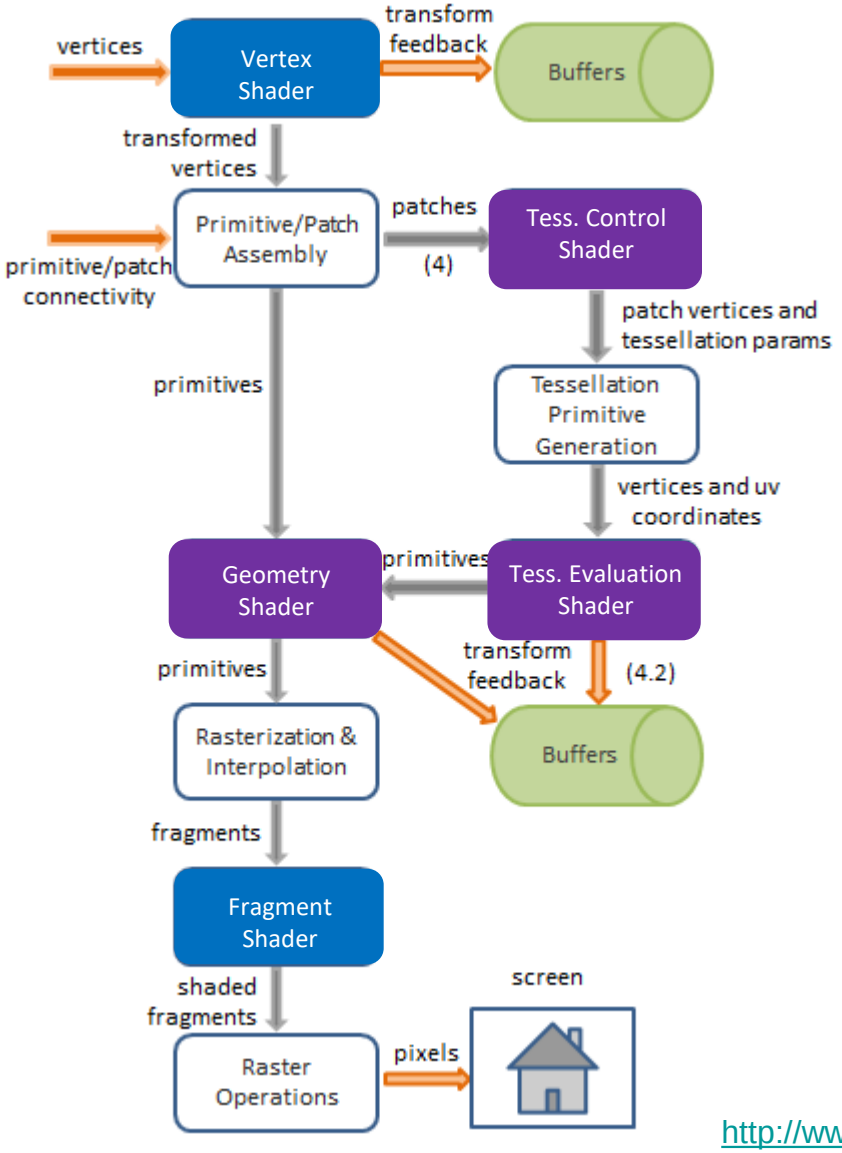
Initialising vec3s in GLM

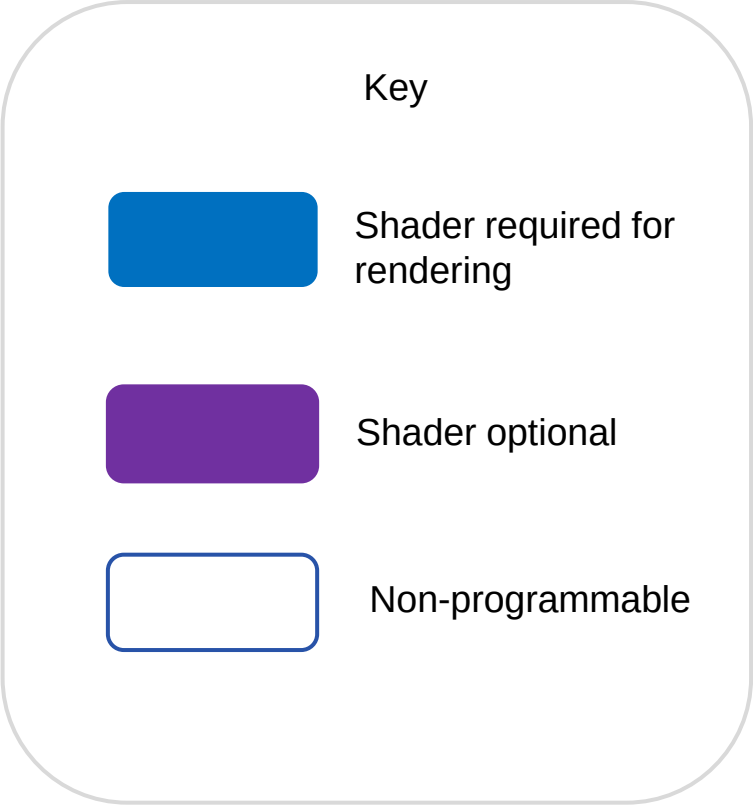
```
glm::vec3 a = glm::vec3(1, 2, 3);

glm::vec3 a(1, 2, 3);
```

Overview of today's lecture

OpenGL basics
Primitives
Matrix mathematics
Transformations and the modelling matrix
Programming matrices with GLM

OpenGL pipeline (less simplified)





http://www.lighthouse3d.com/tutorials/glsl-core-tutorial/pipeline33/

The OpenGL state machine

OpenGL is a *state machine*.

- It has an inherent state represented by internal variables
- Variables initially are in their default state
- Variables changed through OpenGL calls
- The advantage of this implementation is that fewer arguments are passed to OpenGL on each function call

⇨ A common source of programming errors results from the programmer misunderstanding or making incorrect assumptions about the OpenGL state

To *enable* a state variable, one can call

```
glEnable(GLenum capability);
```

To *disable* the capability, one calls

```
glDisable(GLenum capability);
```

One can *test* if a state variable is enabled with

```
glIsEnabled(GLenum capability);
```

OpenGL data types

- OpenGL defines its own data types to make it easier to port code from one platform to another
- Data types map to a specific minimal format on all platforms
- All data types begin with GL

Data type	Minimum bit width	Description
GLboolean	1	true or false boolean
GLbyte	8	Signed 8 bit integer
GLubyte	8	Unsigned 8 bit integer
GLint	32	Signed 32 bit integer
GLuint	32	Unsigned 32 bit integer
GLfloat	32	32 bit floating point
GLclampf	32	32 bit float in the range [0, 1]
(many more)		

Errors

- Internally, OpenGL maintains a set of error flags, each represents different information.
- Whenever an error occurs, the corresponding flag is set and is retained *until retrieved* with `glGetError()`.

Error code	Description
GL_INVALID_ENUM	An enum argument is out of range
GL_INVALID_VALUE	A numeric argument is out of range
GL_INVALID_OPERATION	The operation is illegal in the current state
GL_OUT_OF_MEMORY	Not enough memory is available
GL_NO_ERROR	No error has occurred

Debugging

- `glGetError()` keeps track of errors since
  - OpenGL was initialised; or
  - The last call to `glGetError()`

Source code	*Error flag*
Initialise OpenGL	**GL_NO_ERROR (= 0)**
Good OpenGL calls	**GL_NO_ERROR**
Bad OpenGL call	**GL_INVALID_OPERATION**
Good OpenGL calls	**GL_INVALID_OPERATION**
GLenum retCode = glGetError();	**return GL_INVALID_OPERATION; reset to GL_NO_ERROR**

- When debugging a specific line of code, it is best to make sure the error flag is GL_NO_ERROR before calling the code in question

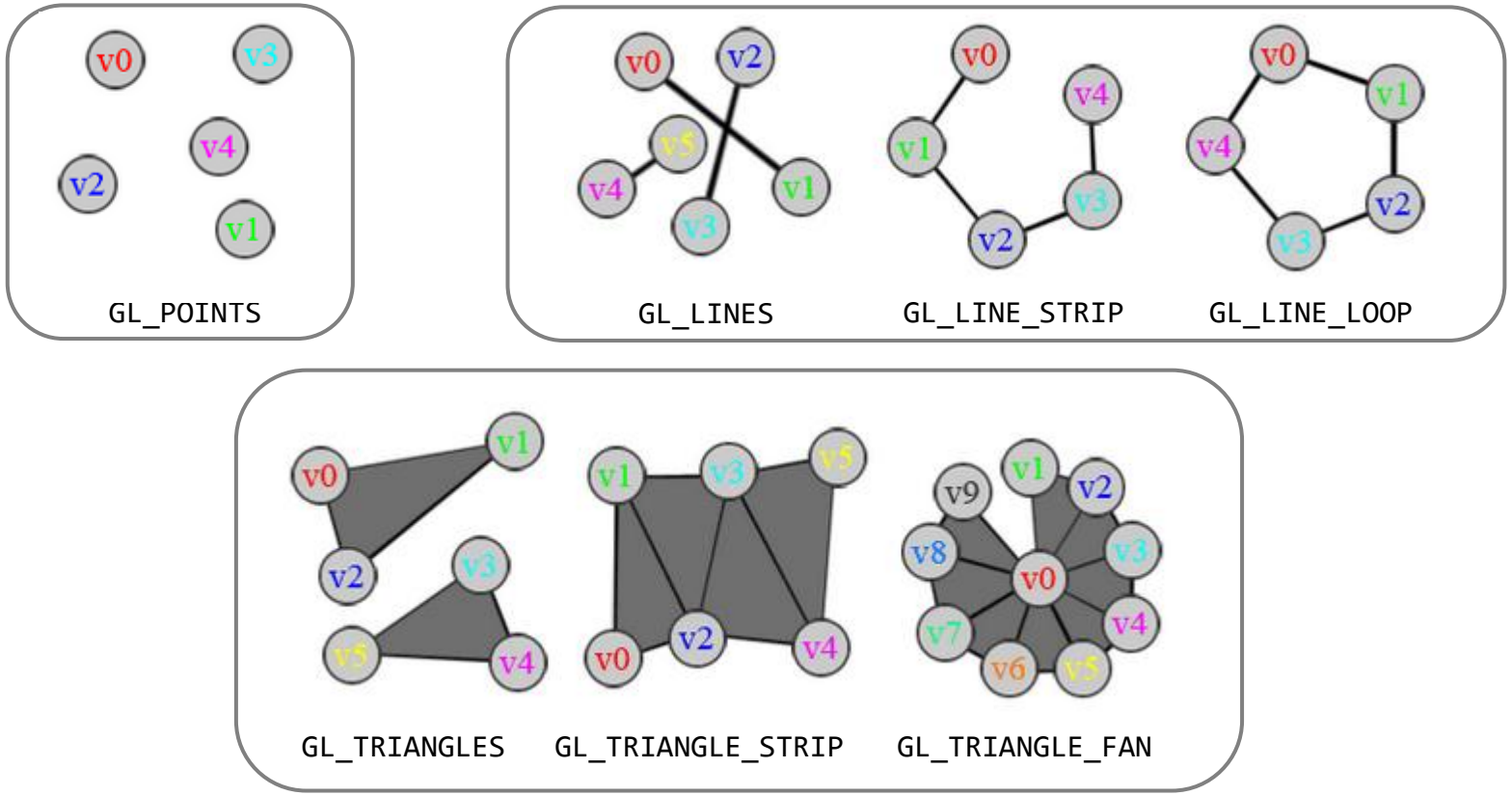
```
glGetError(); // clears the error flag if not GL_NO_ERROR
OpenGL code in question
GLenum retCode = glGetError();
```

Our cast of helper libraries

- **GLEW** (http://glew.sourceforge.net/), the OpenGL Extension Wrangler is used to easily access modern OpenGL functionality.
- **GLM** (http://glm.g-truc.net/), the OpenGL Mathematics library, provides support for vectors and matrices and is designed to be compatible with GLSL, following similar syntax. It provides replacement functions for many deprecated OpenGL calls, particularly those related to matrices and viewing.
- **FreeType** (http://www.freetype.org/) is freely available software library to render fonts. It is used in many games, including World of Warcraft.
- **Open Asset Import Library** (http://assimp.sourceforge.net/) is a portable Open Source library to import various well-known 3D model formats.
- **FreeImage** (http://freeimage.sourceforge.net/) is an open-source library for loading images in common formats like JPG and BMP. It is used in various game engines like Unity and Ogre.

Primitives

- Primitives are the basic geometric entities used by the graphics system. Three basic types: *points*, *lines*, and *triangles**.



- How could one render a quadrilateral (four sided polygon)?

** Note: Geometry and tessellation shaders have additional types*

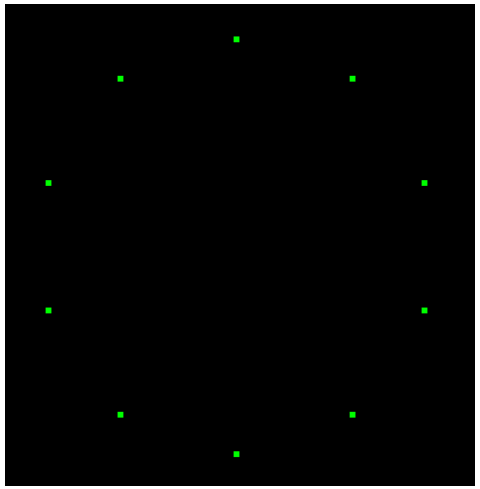
Rendering primitives

- Rendering in modern OpenGL is usually done using:
  - `glDrawArrays (GLenum mode, GLint first, GLsizei count);`

  The vertices comprising the primitive are stored in buffer on the GPU.
- `GLenum mode` is the primitive type, one of
  - `GL_POINTS`
  - `GL_LINES`
  - `GL_LINE_STRIP`
  - `GL_LINE_LOOP`
  - `GL_TRIANGLES`
  - `GL_TRIANGLE_STRIP`
  - `GL_TRIANGLE_FAN`
- `GLint first` is the index to the first element to be rendered
- `Glsizei count` is the number of elements to be rendered

Rendering points

- `GL_POINTS`: Each vertex is a single point on the screen
- Example: rendering 10 green points arranged on a circle



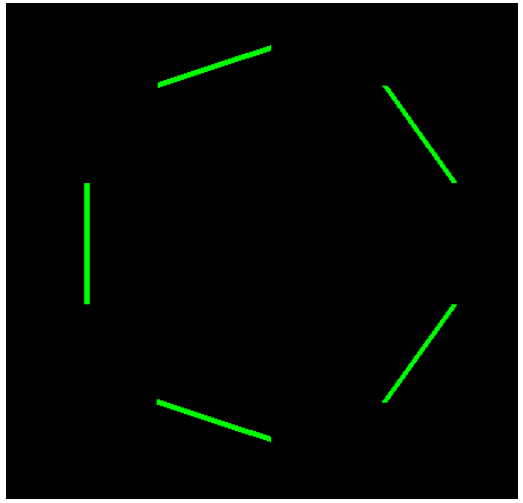
```
glDrawArrays(GL_POINTS, 0, 10);
```

- By default, points are rendered to be one pixel in size. To change this, call

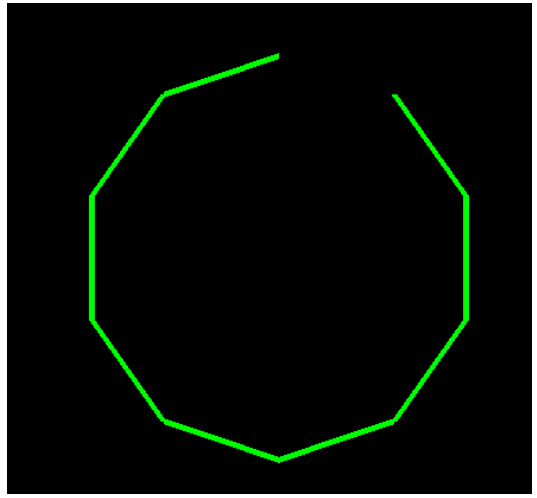
```
glPointSize(GLfloat size);
```

Rendering lines

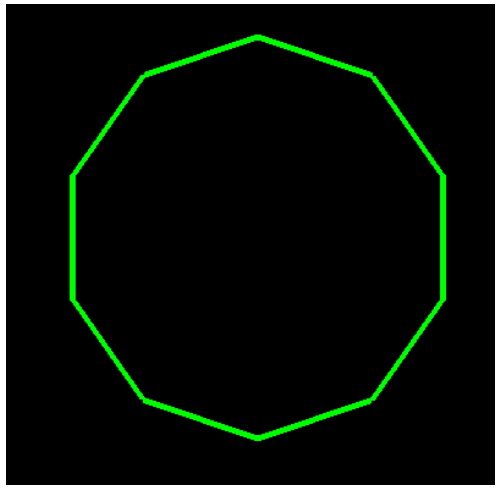
- `GL_LINES`: Each pair of vertices defines a line segment.
- `GL_LINE_STRIP`: A line segment is drawn from the first vertex to each successive vertex.
- `GL_LINE_LOOP`: Same as `GL_LINE_STRIP`, but the last and first vertex are connected.



`GL_LINES`



`GL_LINE_STRIP`



`GL_LINE_LOOP`

- By default, lines are rendered one pixel wide. To change this, call

```
glLineWidth(GLfloat width);
```

Rendering triangles

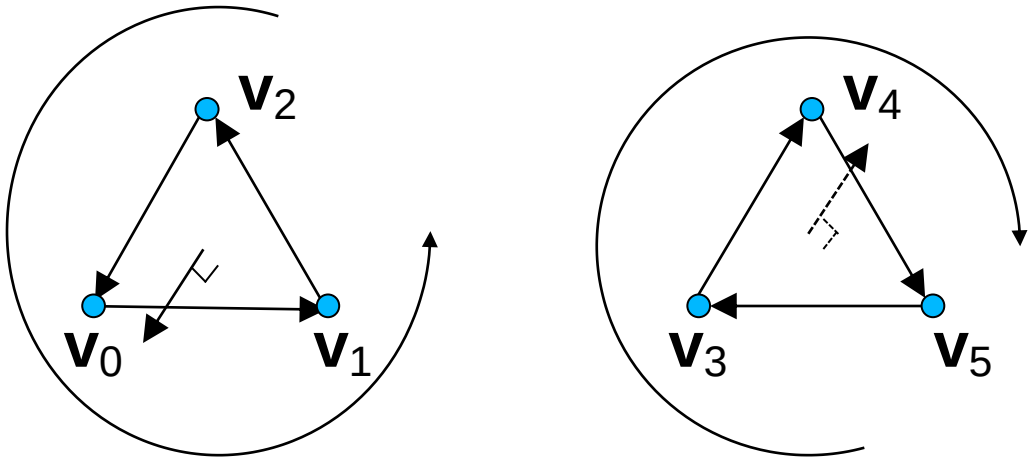
- `GL_TRIANGLES`: Every three vertices define a new triangle.
- `GL_TRIANGLE_STRIP`: Triangles share vertices along a strip.
- `GL_TRIANGLE_FAN`: Triangles fan out from *an origin*, sharing adjacent vertices

⇨ Note: the GPU is *highly* optimised for triangle rendering.

Triangle winding

- Triangles have two sides. The front and back sides are determined using winding, that is, the order in which the vertices are given.

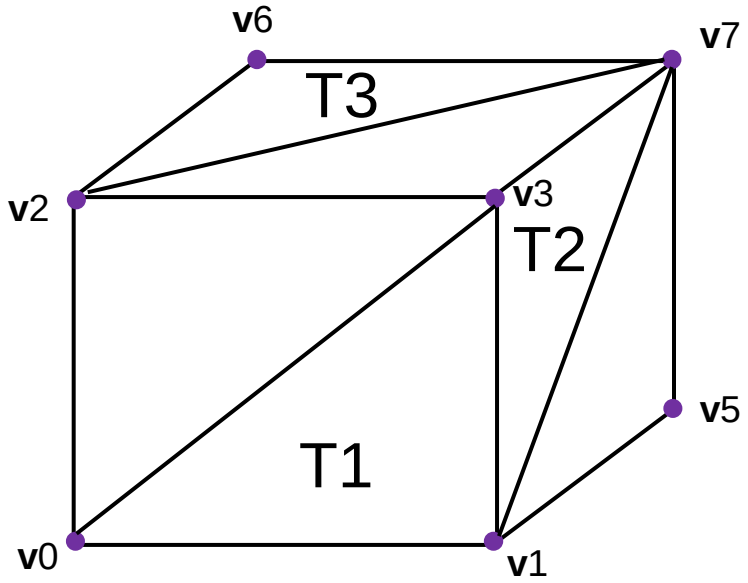
Counter-clockwise
(front facing)



Clockwise
(back facing)

- It is possible to give the front and back sides of the polygon a different appearance, or even not render the back side!
- By default, OpenGL assumes triangles with a counter-clockwise winding are front facing. This can be changed to clockwise using `glFrontFace(GL_CW)`; and back again using `glFrontFace(GL_CCW);`

Question



- For triangle T1, which windings produce a counter-clockwise front face that is outward facing?
  - A: {**v**3, **v**0, **v**1}
  - B: {**v**0, **v**3, **v**1}
  - C: {**v**0, **v**1, **v**3}
  - D: {**v**1, **v**3, **v**0}

Triangles

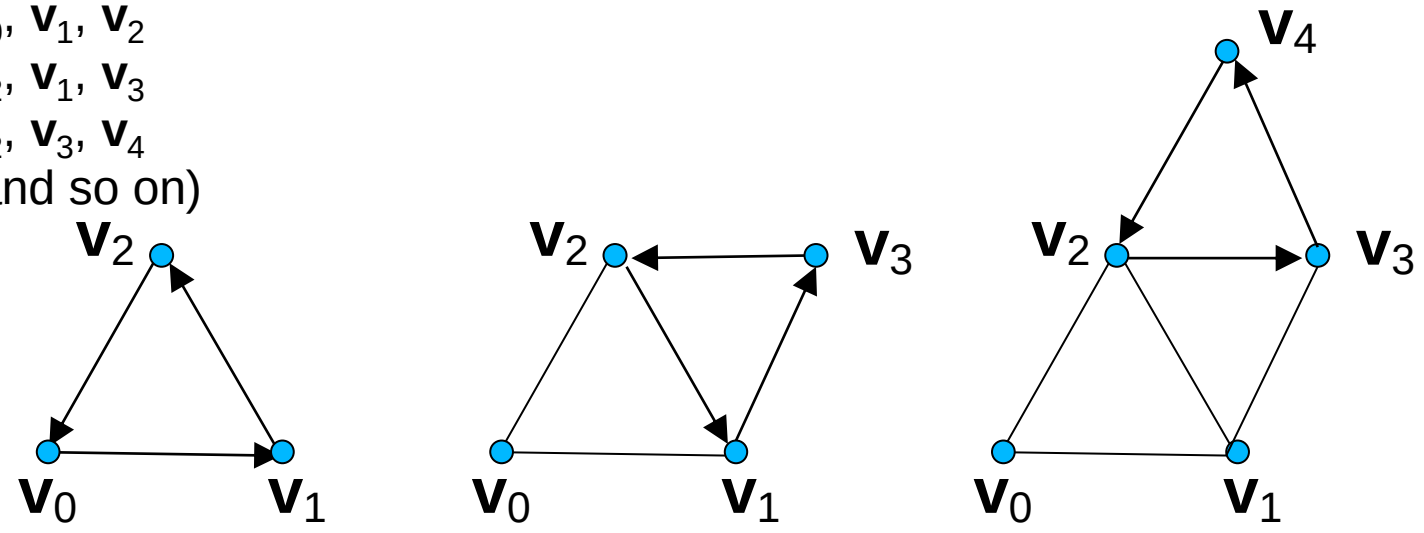
- On the GPU, a set of vertices are stored in a buffer
- When one calls `glDrawArrays` with `GL_TRIANGLES`, every three vertices are rendered as a triangle.

v0.x	triangle 1
v0.y	
v0.z	
v1.x	
v1.y	
v1.z	
v2.x	
v2.y	
v2.z	
v3.x	triangle 2
v3.y	
v3.z	
v4.x	
v4.y	
v4.z	
v5.x	
v5.y	
v5.z	

```
glDrawArrays(GL_TRIANGLES, 0, 6);
```

Triangle strips

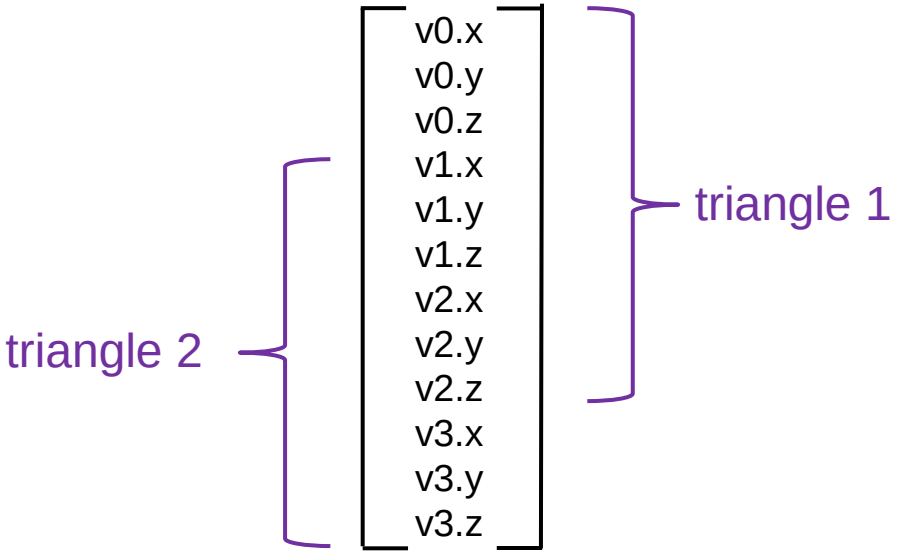
- Triangle strips form a strip of connected triangles.
- To maintain a consistent winding, the vertices are *not* traversed in the order specified; rather the pattern is
  - $\mathbf{v}_0$, $\mathbf{v}_1$, $\mathbf{v}_2$
  - $\mathbf{v}_2$, $\mathbf{v}_1$, $\mathbf{v}_3$
  - $\mathbf{v}_2$, $\mathbf{v}_3$, $\mathbf{v}_4$
  - (and so on)



- The advantage of `GL_TRIANGLE_STRIP` compared to `GL_TRIANGLES`:
  - After the first triangle, specifying the next triangle only requires one additional vertex. This results in less data to store and process.
  - If there are *N* triangles, `GL_TRIANGLES` requires *3N* vertices.
  - How many vertices are required to specify a triangle strip of *N* triangles?
    - Answer: *N+2*

Triangle strips

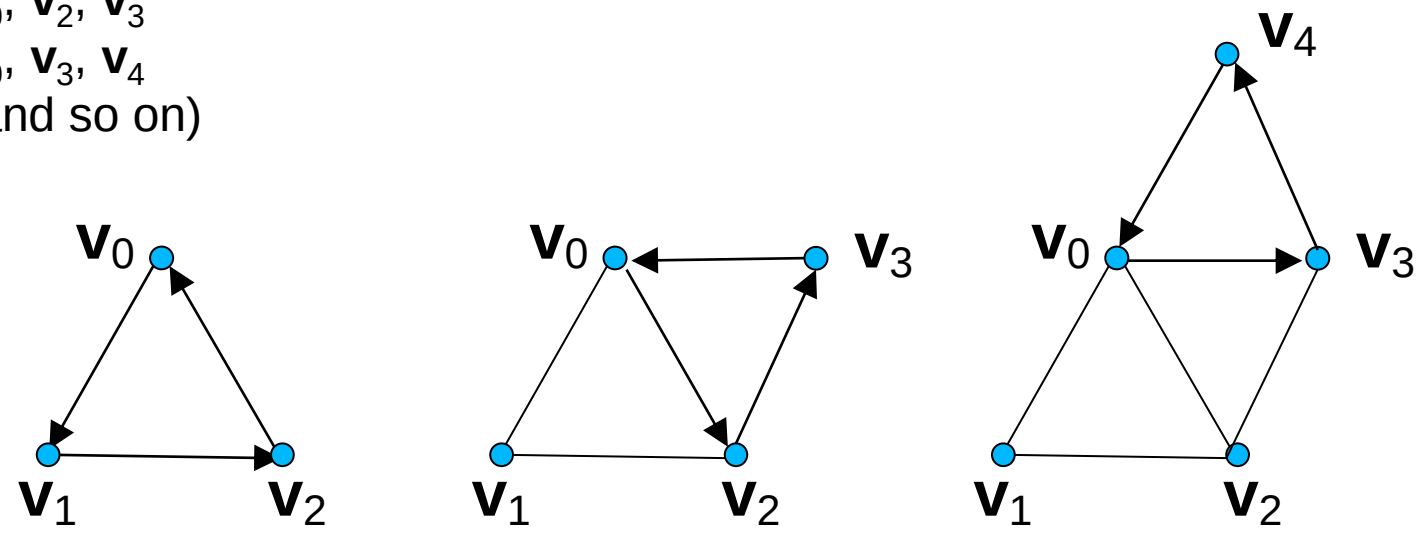
- On the GPU, a set of vertices are stored in a buffer
- When one calls glDrawArrays with GL_TRIANGLE_STRIP, three vertices are rendered as a triangle.




```
glDrawArrays(GL_TRIANGLE_STRIP, 0, 4);
```

Triangle fans

- Triangle fans also form connected triangles, this time around a *central* point given as the first vertex.
- The pattern is
  - $\mathbf{v}_0$, $\mathbf{v}_1$, $\mathbf{v}_2$
  - $\mathbf{v}_0$, $\mathbf{v}_2$, $\mathbf{v}_3$
  - $\mathbf{v}_0$, $\mathbf{v}_3$, $\mathbf{v}_4$
  - (and so on)



- If there are *N* triangles, specifying a triangle fan also requires *N+2* vertices.

Indexed vertices

- Indexed vertices often useful for modeling geometry. They consist of a list of vertices *and* a list of vertex indices.

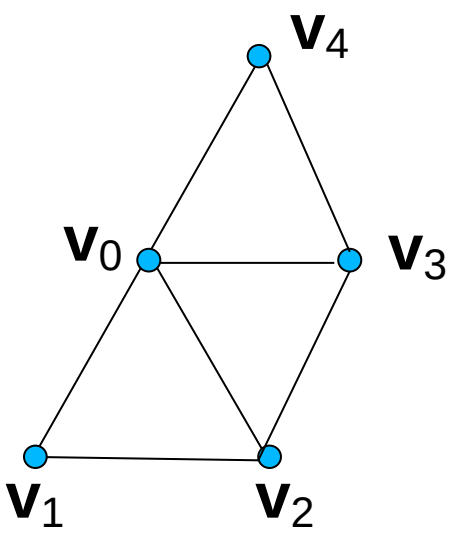
Vertex list

v0.x
v0.y
v0.z
v1.x
v1.y
v1.z
v2.x
v2.y
v2.z
v3.x
v3.y
v3.z
v4.x
v4.y
v4.z

Index list

0	triangle 1
1	
2	
0	triangle 2
2	
3	
0	triangle 3
3	
4	

(Indices for GL_TRIANGLES)



- Indexed geometry is rendered using

```
glDrawElements (mode, count, type, ptrIndices);
```

  - `mode` is the primitive type (GL_TRIANGLES, GL_TRIANGLE_STRIP, etc.)
  - `count` is the number of indices
  - `type` is the datatype of the indices (GL_UNSIGNED_INT)
  - `ptrIndices` is a *pointer* to where the indices are stored (often 0 in the VBO)

Non-indexed vs indexed rendering

- Example three triangles from the last slide, rendered as GL_TRIANGLES

Non-Indexed

Vertex list

v0.x	triangle 1
v0.y	
v0.z	
v1.x	
v1.y	
v1.z	
v2.x	
v2.y	
v2.z	
v0.x	triangle 2
v0.y	
v0.z	
v2.x	
v2.y	
v2.z	
v3.x	
v3.y	
v3.z	
v0.x	triangle 3
v0.y	
v0.z	
v3.x	
v3.y	
v3.z	
v4.x	
v4.y	
v4.z	

Indexed

Vertex list

v0.x
v0.y
v0.z
v1.x
v1.y
v1.z
v2.x
v2.y
v2.z
v3.x
v3.y
v3.z

Index list

0	triangle 1
1	
2	
0	triangle 2
2	
3	
0	triangle 3
3	
4	

Exercise

- You have a vertex list as shown in the table. Draw the output of the following OpenGL calls, assuming a coordinate system with x going to the right, y up, and z out of the page.

Vertex list

-1.0	0.0	0.0
1.0	0.0	0.0
0.0	1.0	0.0
2.0	1.0	0.0
3.0	0.0	0.0

Index list

0
1
2
3
4

```
glDrawElements(GL_TRIANGLE_STRIP, 4, GL_UNSIGNED_INT, (void *)0);
glDrawElements(GL_LINES, 2, GL_UNSIGNED_INT, (void *)(3 *
                    sizeof(GLuint)));
```

From a few slides ago:

`glDrawElements (mode, count, type, indices);`

- `mode` is the primitive type (GL_TRIANGLES, GL_TRIANGLE_STRIP, etc.)
- `count` is the number of indices
- `type` is the datatype of the indices (`GL_UNSIGNED_INT`)
- `indices` is a *pointer* to where the indices are stored

Matrices

- Of fundamental importance in any graphics engine
- A matrix **F** is a rectangular array of numbers (elements) that has N rows and M columns (matrix with size N x M). For example, here is a 3 x 4 matrix:

$$\mathbf{F} = \begin{bmatrix} F_{11} & F_{12} & F_{13} & F_{14} \\ F_{21} & F_{22} & F_{23} & F_{24} \\ F_{31} & F_{32} & F_{33} & F_{34} \end{bmatrix}$$

index

element

- Matrices for which N = M are called *square*.
- Matrices are typically used to represent transformations (e.g., between coordinate systems, rotation, scale, translation, etc.)
- In graphics programming, matrices are often square, with size 3 x 3 or 4 x 4.

Matrices

- A vector is simply a matrix with N or M equal to 1. For example, this is column vector is a 3 x 1 matrix:

$$\mathbf{p} = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix}$$

- Two matrices are equal if and only if all elements are equal. Note this requires the matrices have the same size. For example,

$$\mathbf{A} = \begin{bmatrix} 1 & 3 \\ 3 & 8 \end{bmatrix} \qquad \mathbf{B} = \begin{bmatrix} 8 & 3 \\ 3 & 8 \end{bmatrix} \qquad \mathbf{A} \neq \mathbf{B}$$

Submatrix

- A submatrix is a part of a matrix, formed from taking various rows and columns from a bigger matrix.
- For example, consider a matrix

$$\mathbf{F} = \begin{bmatrix} F_{11} & F_{12} & F_{13} & F_{14} \\ F_{21} & F_{22} & F_{23} & F_{24} \\ F_{31} & F_{32} & F_{33} & F_{34} \end{bmatrix}$$

$$\begin{bmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{bmatrix}$$

upper left 2 x 2 submatrix

$$\begin{bmatrix} F_{23} & F_{24} \\ F_{33} & F_{34} \end{bmatrix}$$

lower right 2 x 2 submatrix

Matrix diagonal

- The matrix diagonal is the set of matrix elements along the diagonal of the (typically square) matrix.
- A diagonal matrix has
  - non-zero elements along the diagonal
  - zero off diagonal
- An example of a diagonal matrix:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 4 \end{bmatrix}$$

Matrix transpose

- Transpose: switches rows with columns

$$F^T_{ij} = F_{ji}$$

$$\mathbf{F} = \begin{bmatrix} F_{11} & F_{12} & F_{13} & F_{14} \\ F_{21} & F_{22} & F_{23} & F_{24} \\ F_{31} & F_{32} & F_{33} & F_{34} \end{bmatrix}$$

$$\mathbf{F}^T = \begin{bmatrix} F_{11} & F_{21} & F_{31} \\ F_{12} & F_{22} & F_{32} \\ F_{13} & F_{23} & F_{33} \\ F_{14} & F_{24} & F_{34} \end{bmatrix}$$

- Addition, subtraction: performed element-wise. Requires matrices to have the same size.

$$\mathbf{F} + \mathbf{G} = \begin{bmatrix} F_{11} + G_{11} & F_{12} + G_{12} & \cdots & F_{1M} + G_{1M} \\ F_{21} + G_{21} & F_{22} + G_{22} & \cdots & F_{2M} + G_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ F_{N1} + G_{N1} & F_{N2} + G_{N2} & \cdots & F_{NM} + G_{NM} \end{bmatrix}$$

Scalar / matrix multiplication

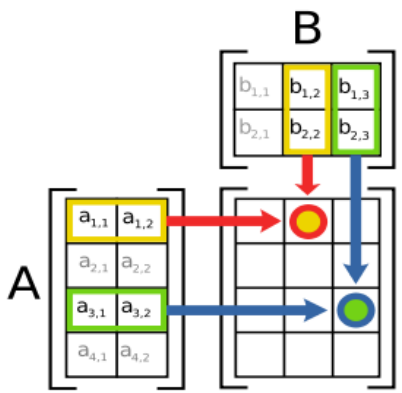
- Scalar / matrix multiplication multiplies each element with scalar

$$a\mathbf{F} = \mathbf{F}a = \begin{bmatrix} aF_{11} & aF_{12} & \cdots & aF_{1M} \\ aF_{21} & aF_{22} & \cdots & aF_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ aF_{N1} & aF_{N2} & \cdots & aF_{NM} \end{bmatrix}$$

Matrix multiplication

- Two matrices can be multiplied together **only** when the number of columns in the matrix on the left equals the number of rows in the matrix on the right.
- For example, suppose
  - matrix A is N x M
  - matrix B is L x R
  - ⇨ The matrix product **AB** is only defined in M = L. The resulting matrix will be of size N x R.
- It is performed by multiplying each row of A by each column of B

$$[\mathbf{AB}]_{ij} = \sum_{k=1}^{M} A_{ik} B_{kj}$$



Matrix multiplication example

- Determine **AB**

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \longleftarrow \mathbf{B}$$

$$\begin{bmatrix} \\ \\ \end{bmatrix}$$

A

Matrix multiplication example

- Multiplication example

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} (1)(5) + (2)(7) & \\ & \end{bmatrix}$$

Matrix multiplication example

- Multiplication example

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \qquad \mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} (1)(5)+(2)(7) & (1)(6)+(2)(8) \\ & \\ & \end{bmatrix}$$

Matrix multiplication example

- Multiplication example

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \qquad \mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} (1)(5)+(2)(7) & (1)(6)+(2)(8) \\ (3)(5)+(4)(7) & \end{bmatrix}$$

Matrix multiplication example

- Multiplication example

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \qquad \mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} (1)(5)+(2)(7) & (1)(6)+(2)(8) \\ (3)(5)+(4)(7) & (3)(6)+(4)(8) \end{bmatrix}$$

Matrix multiplication example

- Multiplication example

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} (1)(5) + (2)(7) & (1)(6) + (2)(8) \\ (3)(5) + (4)(7) & (3)(6) + (4)(8) \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Matrix multiplication not commutative

- One point to be aware of is that matrix multiplication generally does not commute. That is,

$$\mathbf{AB} \neq \mathbf{BA}$$

- For example,

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}\begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae+bg & \dots \\ \dots & \dots \end{bmatrix} \quad \begin{bmatrix} e & f \\ g & h \end{bmatrix}\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} ea+fc & \dots \\ \dots & \dots \end{bmatrix}$$

- Later when we discuss transformations, we'll see the order in which the transformations are done is important, and that ***performing the same transformations in a different order can (and often does) produce a different result***.

Pre- and post-multiplication

- There are terms for describing the order of matrix multiplication
  - When a matrix appears on the left, it is *pre-multiplied.*
  - When a matrix appears on the right, it is *post-multiplied.*
- For example, in the matrix product **AB**,
  - **A** is pre-multiplied to **B**
  - **B** is post-multiplied to **A**

Identity matrix

- In scalar multiplication, any number multiplied with 1 results in the original number.
- Similarly, there is a special square matrix, called the *identity matrix*, which when multiplied by another matrix **A**, results in **A**.
- The identity matrix has the form

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}$$

- For a square matrix **A**, **AI** = **IA** = **A**

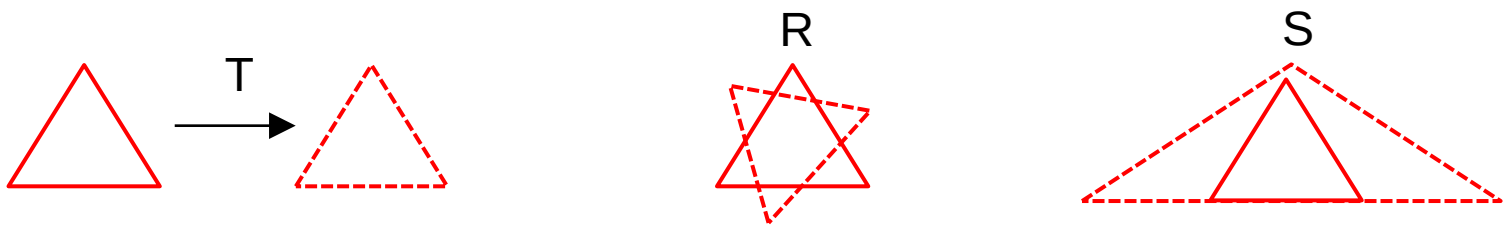
Inverse matrix

- Also in scalar multiplication, multiplication (1/a)*a = 1; that is, multiplication of a number by its inverse gives identity.
- Similarly, multiplication a matrix by its inverse gives the identity matrix.
- The inverse of a matrix **A** is denoted $\mathbf{A}^{-1}$.
  - $\mathbf{AA}^{-1}=\mathbf{I}$, and $\mathbf{A}^{-1}\mathbf{A}=\mathbf{I}$
- If the inverse exists, the matrix is said to be *invertible*. Note: not all matrices are invertible!
- The inverse matrix can be used to provide an inverse transformation.

$$\mathbf{A}=\begin{bmatrix} 9 & -2 \\ -3 & -4 \end{bmatrix} \quad \mathbf{A}^{-1}=\begin{bmatrix} \frac{2}{21} & \frac{-1}{21} \\ \frac{-1}{14} & \frac{-3}{14} \end{bmatrix} \quad \mathbf{AA}^{-1}=\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Transformations

- Matrices are commonly used to transform objects in space. This is achieved by multiplying the matrix by each vertex of the object.
- Common transformations include translation, rotation, and scaling:
  - *Translation*: Moving an object to a new location
  - *Rotation*: Rotating an object about a vector
  - *Scaling*: Used to stretch, shrink, or mirror an object



Modeling matrix

- Earlier versions of OpenGL had fixed functions for matrix mathematics. These functions are no longer part of the OpenGL core.
- Modern OpenGL programming requires that the programmer handle matrices on their own, and pass matrices to shader programs.
- Fortunately, there are external libraries such as GLM to perform useful matrix operations needed for graphics.
- The **modeling matrix** is used to transform objects to a desired location, orientation, and size. This matrix has a size 4 x 4.

Identity matrix

- The identity matrix is a matrix with ones along the *diagonal* and zeros otherwise.

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- When multiplied to a point, the identity matrix does not change the point. For this reason, the identity matrix is the default matrix. For example, let **p** = [x, y, z]^T. We convert this to *homogeneous coordinates* by adding a one to the vector, so **p** = [x, y, z, 1]^T. Notice that **p** = **Ip**.

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

- Homogeneous coordinates add another dimension “w” to a vector, so a point [x, y, z]^T becomes [x, y, z, w]^T. Typically w = 1. Homogeneous coordinates simplify matrix mathematics, making it possible to represent rotation, translation, and scaling using matrices of the same size and with matrix multiplication.

Matrix programming using GLM

- In GLM, methods for 4x4 matrices commonly used in OpenGL include:

translate	rotate	scale	transpose	inverse
ortho	frustum	perspective	lookAt	unProject

- Example usage

```
glm::mat4 a = glm::mat4(1);
glm::mat4 b = glm::translate(a, glm::vec3(1, 0, 0));
glm::mat4 c = glm::rotate(b, 0.0f, glm::vec3(0, 1, 0));
glm::mat4 d = glm::scale(c, glm::vec3(2, 1, 1));
glm::mat4 e = glm::inverse(d);
```

- The modeling matrix is initially set to the identity matrix. In GLM, this can be achieved using

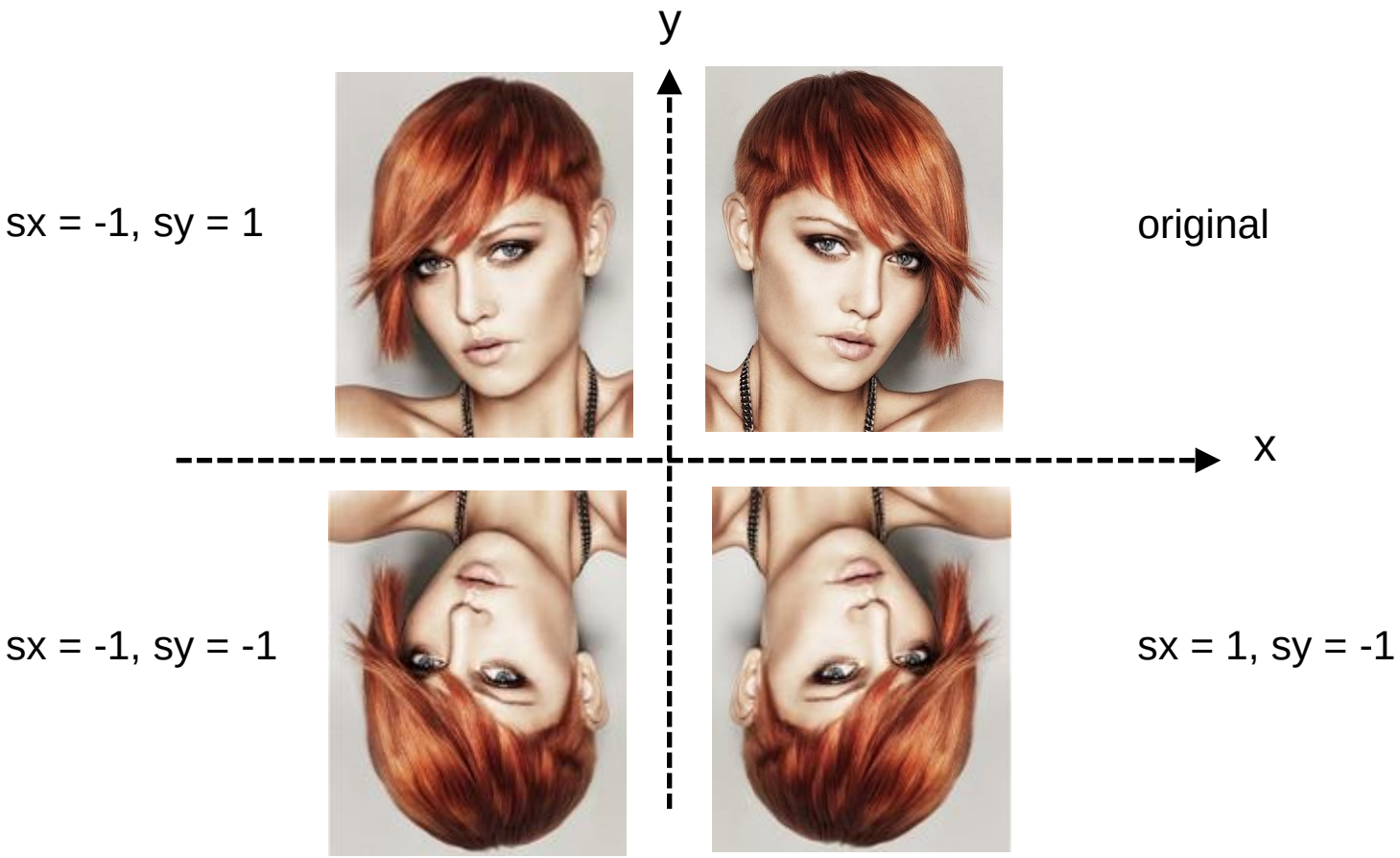
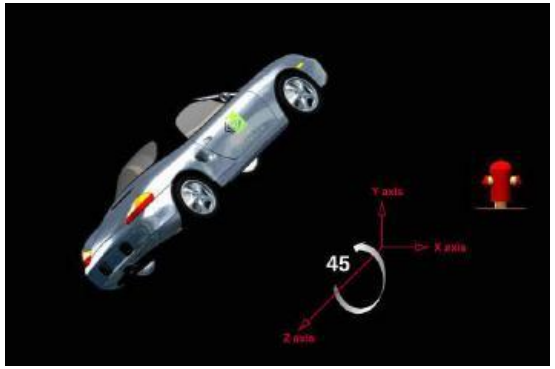
```
glm::mat4 mModelMatrix = glm::mat4(1);
```

Translation

- A translation matrix has the form

$$\mathbf{T}(t_x, t_y, t_z) = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- In GLM, this is programmed with the command
  - ```
    mModelMatrix = glm::translate(mModelMatrix,
                                  glm::vec3(tx, ty, tz));
    ```

    ```

# Translation example

- Suppose we have a point **p** = [x, y, z]$^T$.
- First, we homogenise **p** so that **p** = [x, y, z, 1]$^T$.
- Now, let's determine the matrix product **Tp**:

$$\mathbf{Tp} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ z + t_z \\ 1 \end{bmatrix}$$

# Scaling

- A scaling matrix has the form

$$\mathbf{S}(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- In GLM, this is programmed with the command
  - `mModelMatrix = glm::scale(mModelMatrix, glm::vec3(sx, sy, sz));`
  - Applies a separate scaling to each coordinate.

# Scaling example

- Suppose we have a point **p** = [x, y, z]$^T$.
- First, we homogenise **p** so that **p** = [x, y, z, 1]$^T$.
- Now, let's determine the matrix product **Sp**:

$$\mathbf{Sp} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} s_x x \\ s_y y \\ s_z z \\ 1 \end{bmatrix}$$

# Reflections

- Reflections are implemented using negative values in a scaling matrix.




http://www.hair.becomegorgeous.com/stylish_haircuts/asymmetric_haircuts_ideas-1703.html

# Rotation

- Rotation by an angle $\theta$ (in radians) around an axis (unit-length vector) $[u, v, w]^T$ is given by

$$\mathbf{R}(u, v, w, \theta) = \begin{bmatrix} u^2 + (1-u^2)\cos\theta & uv(1-\cos\theta) - w\sin\theta & uw(1-\cos\theta) + v\sin\theta & 0 \\ uv(1-\cos\theta) + w\sin\theta & v^2 + (1-v^2)\cos\theta & vw(1-\cos\theta) - u\sin\theta & 0 \\ uw(1-\cos\theta) - v\sin\theta & vw(1-\cos\theta) + u\sin\theta & w^2 + (1-w^2)\cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- In GLM, this is programmed with the command
  - ```
    mModelMatrix = glm::rotate(mModelMatrix, angle,
                               glm::vec3(x, y, z));
    ```
  - Angle is expressed in *degrees*; rotation is counter-clockwise around a vector $[x, y, z]^T$.


    ```

Rotation about specific axes

- Rotation around the *x*, *y*, and *z* axes is given by

$$\mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \mathbf{R}_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \mathbf{R}_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- These are special cases of the equation on the previous slide.

Rotation example

- Suppose we have a point **p** = [x, y, z]^T.
- First, we homogenise **p** so that **p** = [x, y, z, 1]^T.
- Now, let's determine the matrix product **Rp**, where **R** is a rotation about the *z*-axis.

$$\mathbf{Rp} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x\cos\theta - y\sin\theta \\ x\sin\theta + y\cos\theta \\ z \\ 1 \end{bmatrix}$$

Composite transforms in OpenGL

- First, the modeling matrix is initialised to the identity matrix

$$\boxed{M = I}$$

- Calls to `glm::translate`, `glm::scale`, and `glm::rotate` update the modeling matrix by *post-multiplying* a transformation matrix. For example, suppose we now call `glm::translate`

$$\boxed{M = I\,T}$$

- And now we call `glm::rotate`

$$\boxed{M = I\,T\,R}$$

- Transforming a vertex with the modeling matrix is implemented as a matrix multiplication, but *in the opposite order* the transformations were given (i.e., the last operation specified is the first to be applied)

$$p \longrightarrow \boxed{M} \longrightarrow p' = I\,T\,R\,p$$

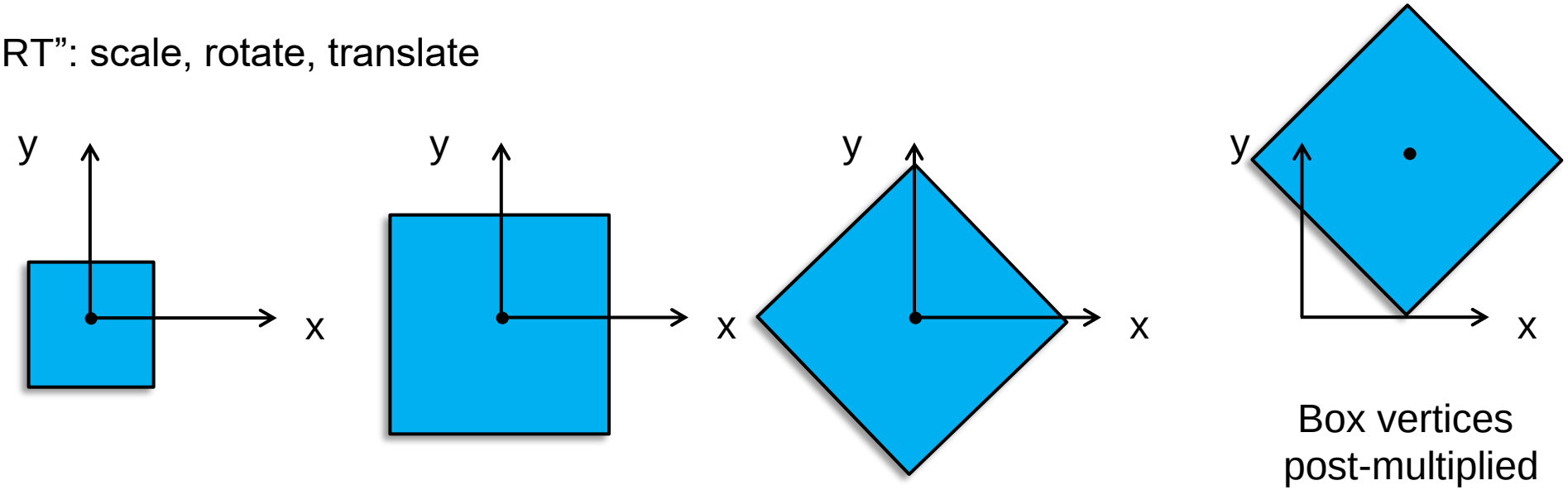
Tip

- Rotations and scales are about the origin
- If your object is centred at the origin, “SoRT” it out:
  - Apply scaling (**S**)
  - Apply rotation (**R**)
  - *Then* apply translation (**T**)
- Since matrices are post-multiplied, the code would look like this:
  - `glm::mat4 modelMatrix = glm::mat4(1);`
  - `modelMatrix = glm::translate(modelMatrix, glm::vec3(tx, ty, tz));`
  - `modelMatrix = glm::rotate(modelMatrix, angle, glm::vec3(x, y, z));`
  - `modelMatrix = glm::scale(modelMatrix, glm::vec3(sx, sy, sz));`

⇨ Note that because of the matrix post-multiplication, the transformations are called *in the opposite order* they are applied!

Example of “SoRTing” it out

- “SoRT”: scale, rotate, translate

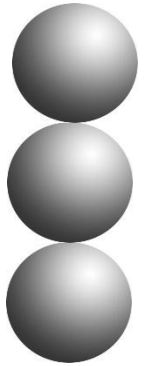


Box vertices
post-multiplied

M = I T R S

```
glm::mat4 modelMatrix = glm::mat4(1);
modelMatrix = glm::translate(modelMatrix, glm::vec3(3, 4, 0));
modelMatrix = glm::rotate(modelMatrix, 45, glm::vec3(0, 0, 1));
modelMatrix = glm::scale(modelMatrix, glm::vec3(2, 2, 2));
drawBox();
```

Relative positioning



- The modeling matrix represents the current transformation
- This provides relative positioning
- For example, suppose you have a function renderSphere, that renders a unit sphere (sphere of radius one centred at the origin) using the current modelview matrix.
- You wish to create a stack of spheres centred at $[0, 0, 0]^T$, $[0, 0, 2]^T$, and $[0, 0, 4]^T$

```
glm::mat4 modelMatrix = glm::mat4(1);
renderSphere();
modelMatrix = glm::translate(modelMatrix, glm::vec3(0, 0, 2));
renderSphere();
modelMatrix = glm::translate(modelMatrix, glm::vec3(0, 0, 2));
renderSphere();
```

⇨ What if now you wish to render something back at the origin?

A matrix stack

- When using matrices, particularly the modeling matrix, it is often helpful to save and restore the matrix during rendering.
- The *matrix stack* allows one to save the current matrix, perform some transformations, and then return to the saved matrix.
  - A **push** will save the current matrix on the stack.
  - A **pop** will restore the previously saved matrix.
- A matrix stack is part of `glUtil`, designed to work with GLM

glutil::MatrixStack

- `glutil::MatrixStack` supports the following methods

SetIdentity	**Translate**	**Rotate**	**RotateX**
RotateY	**RotateZ**	**Push**	**Pop**
Scale	**Top**	**(more)**	

```
glutil::MatrixStack modelMatrixStack;
modelMatrixStack.SetIdentity();  // modeling at the matrix at origin
modelMatrixStack.Push();         // saved the modeling matrix at origin
    renderSphere();
    modelMatrixStack.Translate(glm::vec3(0, 0, 2));
    renderSphere();
    modelMatrixStack.Translate(glm::vec3(0, 0, 2));
    renderSphere();
modelMatrixStack.Pop();          // restored the modeling matrix at origin
```

The matrix at the top of the stack is `modelMatrixStack.Top()`